

# СОДЕРЖАНИЕ

|                                                                                                      |           |
|------------------------------------------------------------------------------------------------------|-----------|
| <b>ВВЕДЕНИЕ . . . . .</b>                                                                            | <b>6</b>  |
| <b>1 Аналитический раздел . . . . .</b>                                                              | <b>7</b>  |
| 1.1 Анализ предметной области . . . . .                                                              | 7         |
| 1.2 Сравнительный анализ существующих решений . . . . .                                              | 7         |
| 1.3 Диаграмма прецедентов разрабатываемого комплекса . . . . .                                       | 9         |
| 1.4 Формализация и описание информации, подлежащей хранению<br>разрабатываемым комплексом . . . . .  | 9         |
| 1.5 Анализ подходов к организации хранилища информации . . . . .                                     | 10        |
| 1.6 Анализ существующих баз данных на основе формализации<br>данных . . . . .                        | 11        |
| 1.6.1 Дореляционные базы данных . . . . .                                                            | 12        |
| 1.6.2 Реляционные базы данных . . . . .                                                              | 12        |
| 1.6.3 Постреляционные базы данных . . . . .                                                          | 13        |
| 1.6.4 Вывод . . . . .                                                                                | 13        |
| 1.7 Анализ существующих типов систем управления базами данных<br>(СУБД) по способу доступа . . . . . | 13        |
| 1.7.1 Файл-серверные СУБД . . . . .                                                                  | 14        |
| 1.7.2 Клиент-серверные СУБД . . . . .                                                                | 14        |
| 1.7.3 Встраиваемые СУБД . . . . .                                                                    | 15        |
| 1.7.4 Сравнительный анализ рассмотренных СУБД . . . . .                                              | 15        |
| 1.8 Выводы . . . . .                                                                                 | 16        |
| <b>2 Конструкторский раздел . . . . .</b>                                                            | <b>17</b> |
| 2.1 Архитектура разрабатываемого программно-алгоритмического<br>комплекса . . . . .                  | 17        |
| 2.2 Взаимодействие компонентов программно-алгоритмического<br>комплекса . . . . .                    | 18        |
| 2.3 Ключевые структуры данных . . . . .                                                              | 18        |
| 2.4 Схема алгоритма монтирования устройства . . . . .                                                | 19        |
| 2.5 Схема алгоритма удаления устройства из списка доверенных<br>устройств . . . . .                  | 21        |

|          |                                                                                                                       |           |
|----------|-----------------------------------------------------------------------------------------------------------------------|-----------|
| 2.6      | Схема алгоритма обработки узлов устройств и записей о точках<br>монтажирования при инициализации приложения . . . . . | 22        |
| 2.7      | Схема алгоритма проверки записи о монтажировании на предмет<br>актуальности . . . . .                                 | 23        |
| 2.8      | Описание сущностей проектируемой базы данных . . . . .                                                                | 25        |
| 2.9      | Диаграмма проектируемой базы данных . . . . .                                                                         | 25        |
| 2.10     | Описание проектируемых ограничений целостности базы данных                                                            | 26        |
| <b>3</b> | <b>Технологический раздел . . . . .</b>                                                                               | <b>28</b> |
| 3.1      | Выбор языка и среды программирования . . . . .                                                                        | 28        |
| 3.2      | Выбор средств тестирования кода . . . . .                                                                             | 29        |
| 3.3      | Обеспечение связи между клиентом и сервером . . . . .                                                                 | 29        |
| 3.4      | Выбор программных средств для работы с устройствами и фай-<br>ловыми системами . . . . .                              | 30        |
| 3.5      | Выбор встраиваемой системы управления реляционными базами<br>данных (СУБД) . . . . .                                  | 30        |
| 3.6      | Классы эквивалентности . . . . .                                                                                      | 31        |
| 3.6.1    | Классы эквивалентности для метода построения имени<br>папки монтажирования . . . . .                                  | 31        |
| 3.6.2    | Классы эквивалентности для обработки события подклю-<br>чения устройства . . . . .                                    | 32        |
| 3.6.3    | Классы эквивалентности для получения записи о монти-<br>ровании по узлу устройства . . . . .                          | 32        |
| 3.6.4    | Классы эквивалентности для получения списка доверен-<br>ных устройств . . . . .                                       | 32        |
| 3.7      | Структура проекта серверной части комплекса . . . . .                                                                 | 33        |
| 3.8      | Листинги ключевых методов, классов и функций . . . . .                                                                | 37        |
| 3.9      | Развёртывание приложения . . . . .                                                                                    | 42        |
| 3.10     | Взаимодействие пользователя с приложением . . . . .                                                                   | 44        |
| 3.10.1   | Экран подключённых устройств . . . . .                                                                                | 44        |
| 3.10.2   | Экран доверенных устройств . . . . .                                                                                  | 45        |
| <b>4</b> | <b>Исследовательский раздел . . . . .</b>                                                                             | <b>47</b> |
| 4.1      | Постановка исследования . . . . .                                                                                     | 47        |
| 4.2      | Технические характеристики . . . . .                                                                                  | 47        |

|                                                   |                                                |           |
|---------------------------------------------------|------------------------------------------------|-----------|
| 4.3                                               | Проведение исследования и результаты . . . . . | 47        |
| 4.3.1                                             | План первой части исследования . . . . .       | 47        |
| 4.3.2                                             | Результаты первой части исследования . . . . . | 48        |
| 4.3.3                                             | План второй части исследования . . . . .       | 49        |
| 4.3.4                                             | Результаты второй части исследования . . . . . | 49        |
| <b>ЗАКЛЮЧЕНИЕ . . . . .</b>                       |                                                | <b>51</b> |
| <b>СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ . . . . .</b> |                                                | <b>55</b> |
| <b>ПРИЛОЖЕНИЕ А . . . . .</b>                     |                                                | <b>56</b> |

## ВВЕДЕНИЕ

Использование внешних запоминающих устройств создаёт риск попадания на компьютер вредоносного программного обеспечения и увеличивает вероятность несанкционированного копирования информации на сторонние носители.

Цель данной работы — разработка и реализация программно-алгоритмического комплекса, снижающего риски использования внешних запоминающих устройств.

Для достижения поставленной цели необходимо решить следующие задачи:

- проанализировать существующие решения;
- формализовать требования к разрабатываемому комплексу;
- разработать архитектуру программно-алгоритмического комплекса;
- обосновать выбор средств программной реализации;
- разработать программное обеспечение комплекса;
- провести тестирования комплекса;
- исследовать характеристики разработанного программного обеспечения.

# **1 Аналитический раздел**

В данном разделе анализируется предметная область, существующие решения, формализуются требования к разрабатываемому комплексу. Также приводится описание информации, используемой программным комплексом в процессе работы и анализируются способы её хранения.

## **1.1 Анализ предметной области**

Наиболее часто встречающимся интерфейсом для подключения внешних запоминающих устройств к вычислительной технике является USB (Universal Serial Bus). К таким устройствам относятся флеш-карты и жёсткие диски. Широкое использование USB-устройств для хранения и передачи данных обусловлено их универсальностью, и надёжностью. В то же время USB-устройства являются одними из наиболее опасных и активно используемых средств и каналов реализации угроз ИБ (информационной безопасности) [1]. О значительном росте (более чем в два раза) инцидентов ИБ в России, связанных с утечками конфиденциальной информации посредством USB-носителей, сообщается в аналитическом отчете отечественной компании InfoWatch [2]. Для противодействия этим угрозам ИБ используются два подхода. Первый подход предполагает полный запрет и отключение физической возможности использования USB-устройств. Реализация данного решения целесообразна для систем, хранящих и обрабатывающих информацию, содержащую сведения, составляющие государственную тайну или особо конфиденциальные данные. Второй подход основан на использовании программных и аппаратно-программных средств защиты от несанкционированного доступа (СЗИ НСД) и реализации соответствующих организационно-технических мер.

## **1.2 Сравнительный анализ существующих решений**

Существующие программные средства защиты от утечки и утери данных условно можно разделить на две основные категории: корпоративные DLP (Data Leak Prevention)-системы и решения, предназначенные для персонального использования. Корпоративные системы ориентированы на эксплуатацию в инфраструктуре организаций и, как правило, имеют распределённую архитектуру, включающую серверные компоненты, средства централизован-

ного администрирования, мониторинга и управления большим количеством рабочих станций. Такие решения обладают широким набором функций контроля и аудита, однако их использование на отдельном персональном компьютере зачастую является избыточным как с функциональной, так и с ресурсной точки зрения. В рамках данного анализа рассматриваются только решения, предназначенные для личного использования.

Для анализа были выбраны следующие программные комплексы:

1. USBGuard [3];
2. USBFilter [4];
3. USBWall [5];
4. USBShield [6];
5. Ratool [7];
6. GiliSoft USB Lock [8];
7. USB Block [9].

В таблице 1.1 представлены результаты анализа существующих решений.

Таблица 1.1 – Результат анализа существующих решений

| <b>Критерий</b>                                                   | <b>Номер решения</b> |          |          |          |          |          |          |
|-------------------------------------------------------------------|----------------------|----------|----------|----------|----------|----------|----------|
|                                                                   | <b>1</b>             | <b>2</b> | <b>3</b> | <b>4</b> | <b>5</b> | <b>6</b> | <b>7</b> |
| поддержка ОС (операционной системы) Linux Ubuntu                  | +                    | +        | +        | -        | -        | -        | -        |
| возможность создания списка доверенных устройств                  | +                    | +        | +        | +        | -        | +        | +        |
| возможность установки даты истечения периода доверия к устройству | -                    | -        | -        | -        | -        | -        | -        |
| наличие графического интерфейса                                   | -                    | -        | -        | +        | +        | +        | +        |
| наличие обновлений и поддержки                                    | +                    | -        | -        | +        | +        | +        | +        |

Таким образом, представленные решения не соответствуют в полной мере поставленным критериям.

### 1.3 Диаграмма прецедентов разрабатываемого комплекса

На рисунке 1.1 представлена диаграмма прецедентов разрабатываемого комплекса.



Рисунок 1.1 – Диаграмма прецедентов комплекса

### 1.4 Формализация и описание информации, подлежащей хранению разрабатываемым комплексом

Формальное описание устройства может быть представлено следующими атрибутами:

1. идентификатор производителя;
2. идентификатор устройства;
3. уникальный серийный номер устройства;
4. наименование производителя;
5. наименование устройства.

Доверенное устройство характеризуется формальным описанием устройства и датой истечения доверия.

Запись о монтировании устройства описывается следующей информацией:

1. формальное описание устройства;
2. узел устройства;
3. путь монтирования;
4. режим монтирования («Только чтение» или «Чтение и запись»).

## **1.5 Анализ подходов к организации хранилища информации**

Наиболее распространёнными подходами к организации хранилища данных, используемых программным комплексом, являются использование баз данных и хранение информации в плоских файлах.

В таблице 1.2 представлены результаты сравнительного анализа способов организации хранилищ данных.



Таблица 1.2 – Результат анализа подходов к хранению информации

| <b>Критерий</b>                                                                                                           | <b>Решение</b>           |                        |
|---------------------------------------------------------------------------------------------------------------------------|--------------------------|------------------------|
|                                                                                                                           | <b>Плоские<br/>файлы</b> | <b>Базы<br/>данных</b> |
| наличие встроенных механизмов поддержания целостности данных (ограничения, уникальность)                                  | нет                      | да                     |
| устойчивость хранилища к сбоям и некорректному завершению работы                                                          | нет                      | да                     |
| отсутствие дополнительных зависимостей у прикладного приложения                                                           | да                       | нет                    |
| отсутствие возможности несанкционированного ручного просмотра и редактирования данных без специализированных инструментов | нет                      | да                     |
| возможность выполнения выборок и фильтрации данных без дополнительной реализации алгоритмов поиска                        | нет                      | да                     |
| отсутствие необходимости инициализации и настройки хранилища перед началом работы                                         | да                       | нет                    |
| возможность расширения структуры данных и добавления новых требований без переработки существующей логики хранения        | нет                      | да                     |

На основе таблицы анализа для разрабатываемого комплекса предпочтительнее использовать базу данных.

## 1.6 Анализ существующих баз данных на основе формализации данных

Базы данных подразделяются на три основных типа по модели данных:

1. дореляционные;
2. реляционные;
3. постреляционные.

### 1.6.1 Дореляционные базы данных

Дореляционные базы данных [10] хранят записи в виде связей предок/потомок. Основными недостатками дореляционных баз данных являются избыточность данных и сложность выполнения запросов. Изменение схемы базы данных крайне ограничена. Целостность данных обеспечивается за счёт жёсткой структуры связей. Пример хранения записей в дореляционных базах представлен на рисунке 1.2.

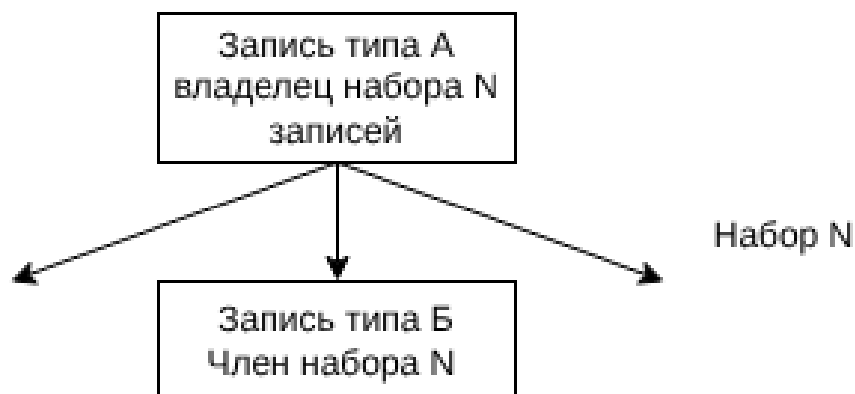


Рисунок 1.2 – Пример хранения записей в дореляционных базах данных

### 1.6.2 Реляционные базы данных

В реляционных базах данных данные хранятся в виде двумерных таблиц. Связь между ними обеспечивается общими столбцами, такими как идентификаторы. Табличная архитектура подходит для хранения структурированной информации. Целостность и непротиворечивость данных соблюдается при помощи ограничений. Пример хранения записей в реляционных базах представлен на рисунке 1.3.

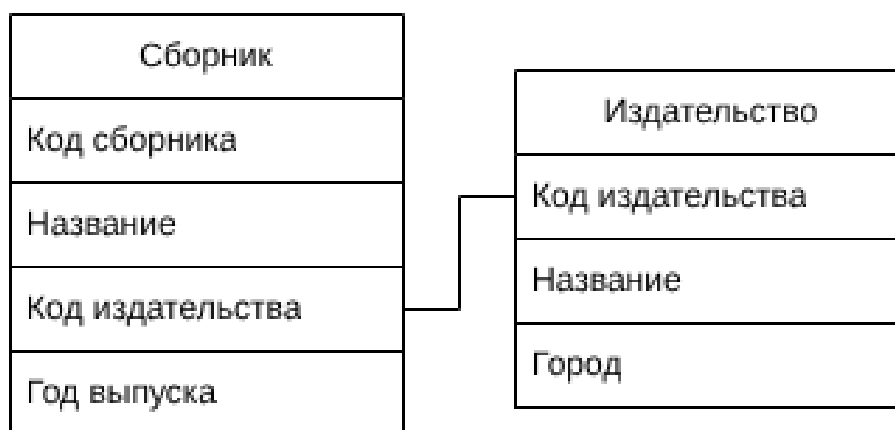


Рисунок 1.3 – Пример хранения записей в реляционных базах данных

### 1.6.3 Постреляционные базы данных

Постреляционные базы данных подходят для хранения неструктурированных данных, поддерживая логические связи между ними. Недостатком является сложность обеспечения целостности данных. Отличаются хорошей горизонтальной масштабируемостью, сложность запросов варьируется в зависимости от конкретной реализации.

### 1.6.4 Вывод

Для реализации поставленной задачи была выбрана реляционная модель, так как она лучше всего подходит для хранения структурированной информации, обеспечивая баланс между целостностью данных, простотой запросов и возможностями масштабирования.

## 1.7 Анализ существующих типов систем управления базами данных (СУБД) по способу доступа

СУБД по способу доступа к базе данных подразделяется на три основных вида [11]:

1. файл-серверные;
2. клиент-серверные;
3. встраиваемые.

### 1.7.1 Файл-серверные СУБД

При работе с файл-серверной СУБД обработка данных происходит на рабочем месте, а сервер применяется только как отдельный накопитель. Каждый пользователь сам использует информацию и вносит изменения в файлы данных и в индексные файлы. На рисунке 1.4 представлена общая схема работы файл-серверных СУБД.

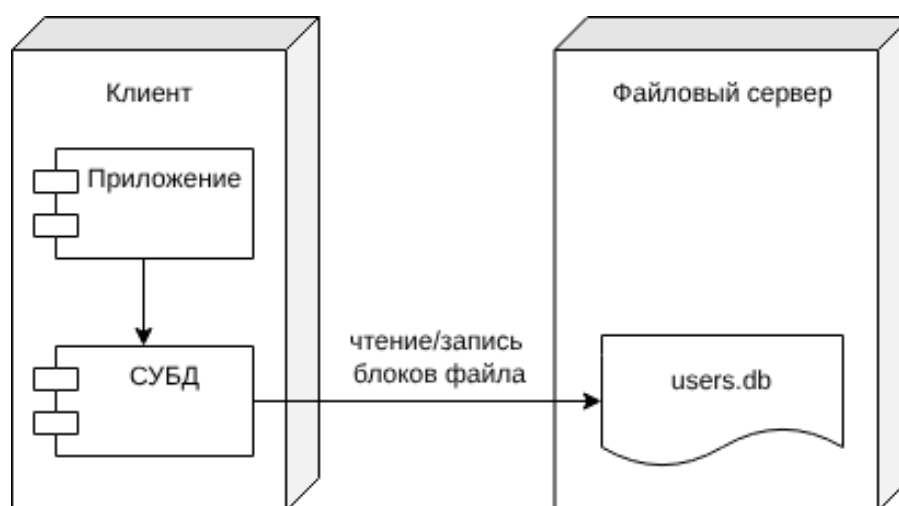


Рисунок 1.4 – Схема работы файл-серверных СУБД

### 1.7.2 Клиент-серверные СУБД

Клиент-серверная СУБД позволяет обмениваться клиенту и серверу минимально необходимыми объёмами информации. Клиент может выполнять функции предварительной обработки перед передачей информации серверу, но в основном его функции заключаются в организации доступа пользователя к серверу. В большинстве случаев клиент-серверная СУБД гораздо менее требовательна к пропускной способности компьютерной сети, чем файл-серверная СУБД, особенно при выполнении операции поиска в базе данных по заданным пользователем параметрам, т.к. для поиска нет необходимости получать на клиент весь массив данных: клиент передаёт параметры запроса серверу, а сервер производит поиск по полученному запросу в локальной базе данных. Результат выполнения запроса возвращается клиенту, который обеспечивает отображение результата пользователю. На рисунке 1.5 представлена общая схема работы клиент-серверных СУБД.

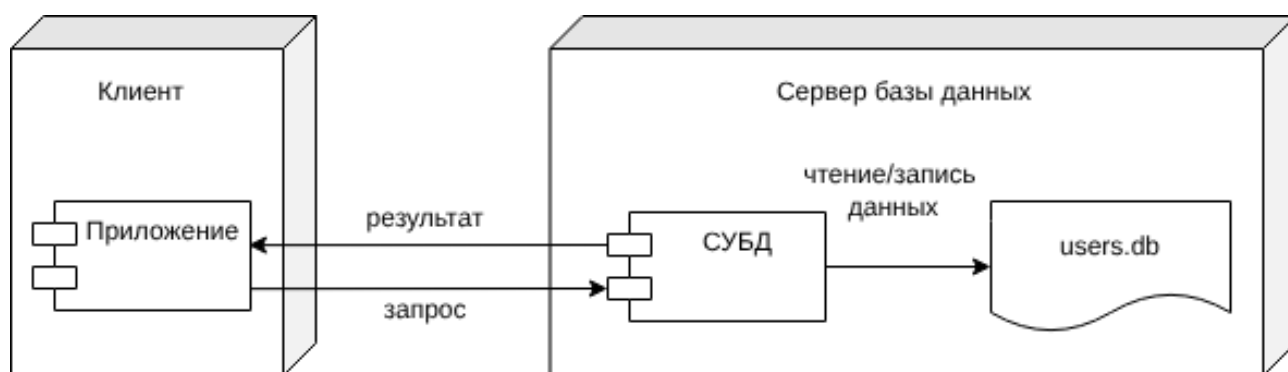


Рисунок 1.5 – Схема работы клиент-серверных СУБД

### 1.7.3 Встраиваемые СУБД

Встраиваемая система управления базой данных — это система, которая может быть связана с клиентским приложением таким образом, чтобы приложение и СУБД работали в едином адресном пространстве. Вместе со встроенной базой данных приложение может быть развернуто как единая программа. Благодаря связыванию приложения с базой данных, прикладная система выигрывает от снижения общей сложности и уменьшения затрат на администрирование. На рисунке 1.6 представлена общая схема работы встраиваемых СУБД.

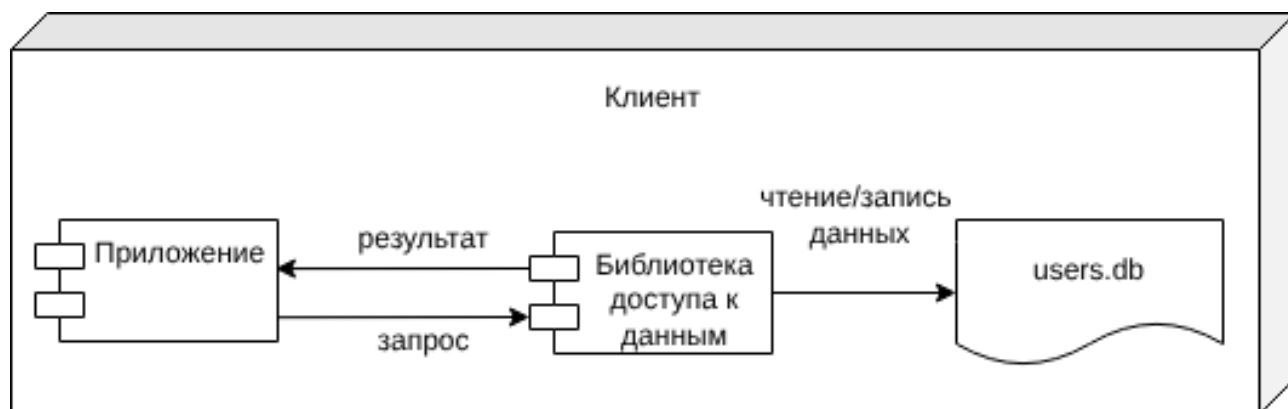


Рисунок 1.6 – Схема работы встраиваемых СУБД

### 1.7.4 Сравнительный анализ рассмотренных СУБД

В таблице 1.3 представлены результаты сравнения рассмотренных СУБД.

Таблица 1.3 – Результаты сравнения СУБД по способу доступа

| <div> <div>Тип СУБД</div> <div>Критерий</div> </div>                    | Файл-серверная | Клиент-серверная | Встраиваемая |
|-------------------------------------------------------------------------|----------------|------------------|--------------|
| Отсутствие избыточности для локального однопользовательского приложения | да             | нет              | да           |
| Отсутствие необходимости сетевого взаимодействия                        | нет            | нет              | да           |
| Отсутствие необходимости в отдельном серверном процессе                 | да             | нет              | да           |

Таким образом, на основании сравнения СУБД, для разрабатываемого программно-алгоритмического комплекса наиболее подходящей будет встраиваемая СУБД.

## 1.8 Выводы

В данном разделе была проанализирована предметная область рисков использования USB-устройств, проведено сравнение существующих решений по сформулированным критериям. Программно-алгоритмический комплекс был формализован с использованием диаграммы прецедентов, описаны представления ключевых структур данных, проанализированы способы организации хранилища, выбраны наиболее подходящие типы базы данных и СУБД.

## 2 Конструкторский раздел

В данном разделе представлена архитектура программно-алгоритмического комплекса, описаны его компоненты, их взаимодействие и ключевые структуры данных, приведены схемы спроектированных алгоритмов, описаны сущности и ограничения целостности проектируемой базы данных.

### 2.1 Архитектура разрабатываемого программно-алгоритмического комплекса

Архитектура комплекса будет представлена моделью клиент-сервер, где клиент — приложение, позволяющее пользователю управлять списком доверенных устройств, а сервер — приложение, отвечающее непосредственно за монтирование устройств, хранение записей о монтировании, работе с базой данных. Схематично данная модель представлена на рисунке 2.1.

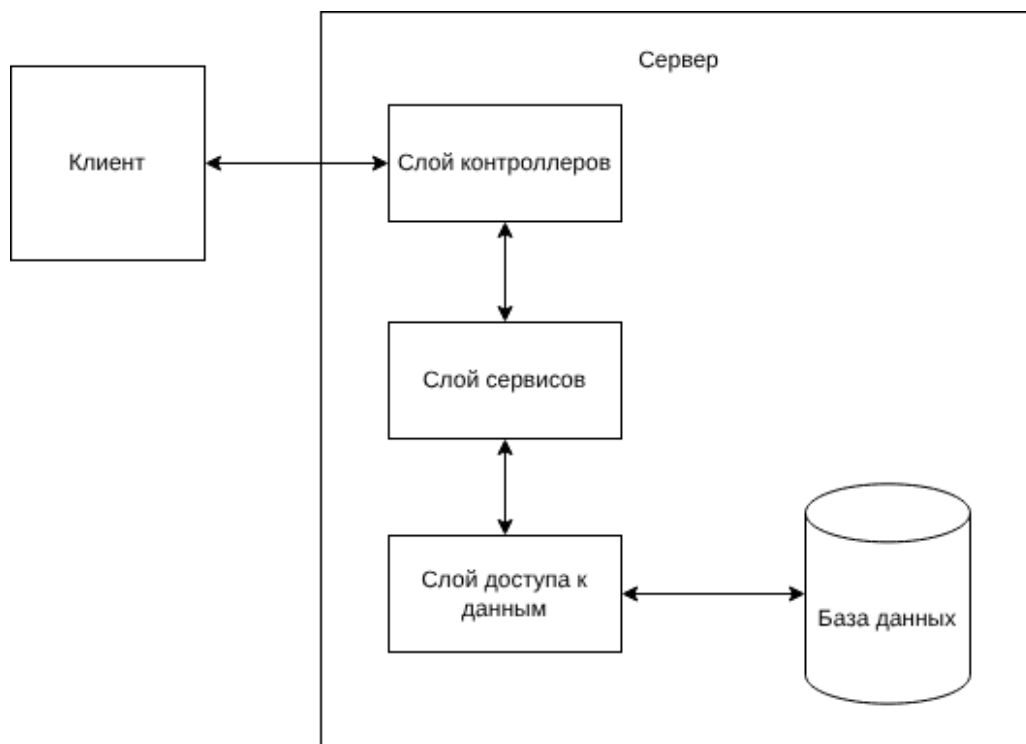


Рисунок 2.1 – Модель клиент-сервер

## 2.2 Взаимодействие компонентов программно-алгоритмического комплекса

Взаимодействие клиента и сервера описывается с помощью спецификации OpenAPI [12]. Использование OpenAPI позволяет формализовать структуру запросов и ответов, определить параметры запросов и форматы передаваемых данных.

На рисунке 2.2 представлено описание программного интерфейса приложения, включающее маршруты взаимодействия клиента и сервера.

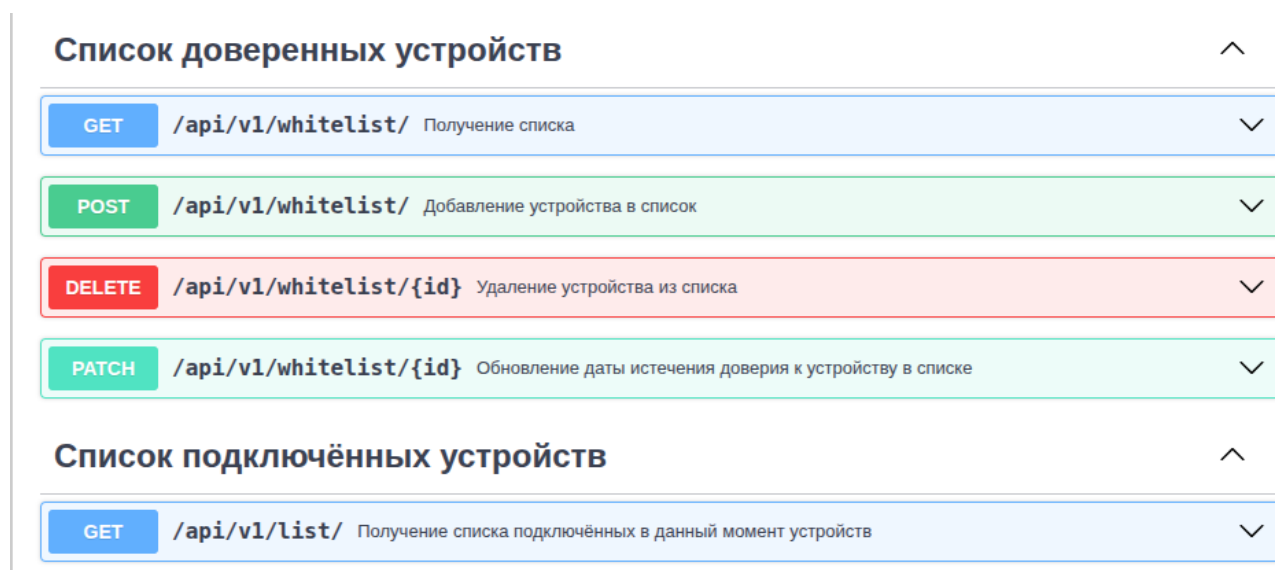


Рисунок 2.2 – Описание программного интерфейса приложения

## 2.3 Ключевые структуры данных

Серверная часть использует несколько структур данных для отправки ответов клиенту и использования во внутренних методах.

— DeviceInfo — структура формального описания USB-устройства:

1. `vendorId` — идентификатор производителя;
2. `productId` — идентификатор устройства;
3. `serial` — серийный номер;
4. `vendorName` — название производителя;
5. `productName` — название продукта.



- **Device** — структура описания доверенного устройства, используется для передачи на клиент при запросе списка доверенных устройств:
  1. **id** — первичный ключ из базы данных;
  2. **info** — формальное описание **DeviceInfo**;
  3. **validTo** — дата истечения доверия.
- **EventType** — перечисляемый тип, используемый для обозначения физического манипулирования с накопителем, где **INSERT(0)** — подключение устройства, **REMOVE(1)** — извлечение устройства;
- **DeviceEvent** — структура, описывающая физическое событие с накопителем. Имеет поля **type** типа **EventType**, и **devNode** — узел устройства;
- **MODE** — перечисляемый тип, используемый для обозначения режима монтирования, где **RO(0)** — режим только для чтения, а **RW(1)** — режим полного доступа;
- **MountRecord** — структура описания события монтирования, используется для передачи на клиент при запросе подключённых в данный момент устройств:
  1. **id** — первичный ключ устройства из базы данных(при наличии);
  2. **devNode** — узел устройства;
  3. **mountPoint** — путь монтирования;
  4. **info** — формальное описание **DeviceInfo**;
  5. **mode** — режим монтирования.

## 2.4 Схема алгоритма монтирования устройства

На рисунке 2.3 представлена схема алгоритма монтирования устройства при его подключении к компьютеру.

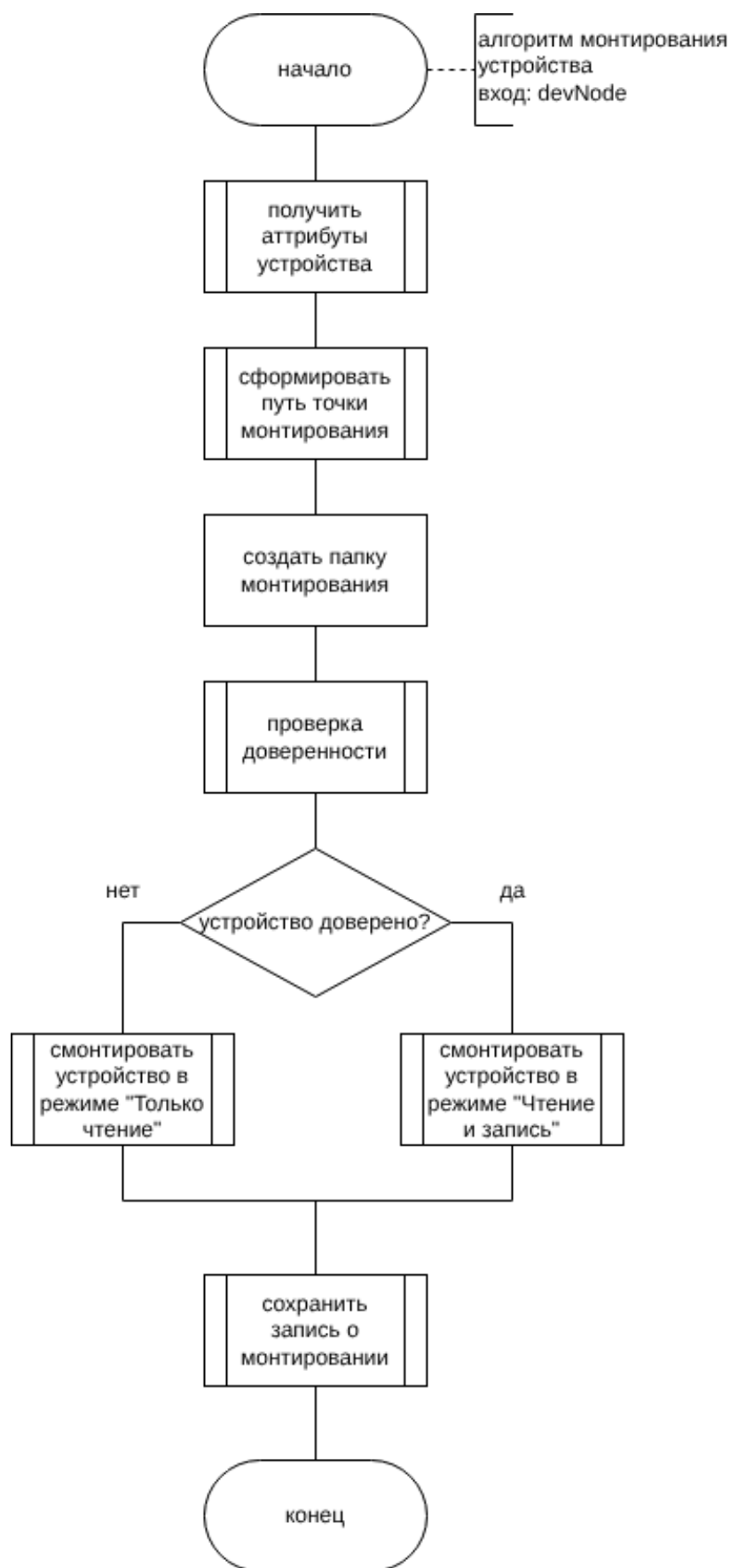


Рисунок 2.3 – Схема алгоритма монтирования

## 2.5 Схема алгоритма удаления устройства из списка доверенных устройств

На рисунке 2.4 представлена схема алгоритма удаления устройства из списка доверенных устройств.

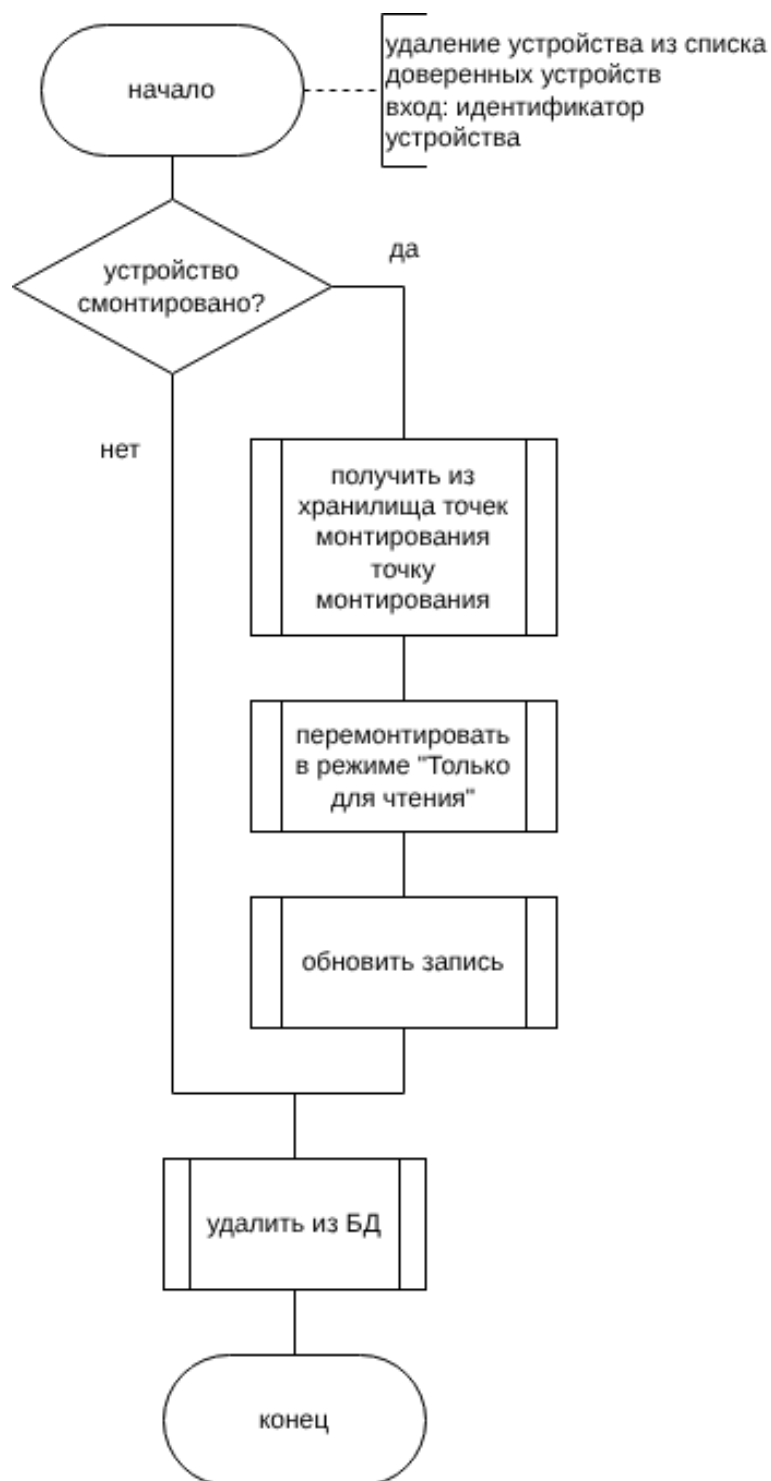


Рисунок 2.4 – Схема алгоритма удаления устройства

## 2.6 Схема алгоритма обработки узлов устройств и записей о точках монтирования при инициализации приложения

На рисунке 2.5 представлена схема алгоритма обработки узлов устройств и записей о точках монтирования при инициализации приложения.

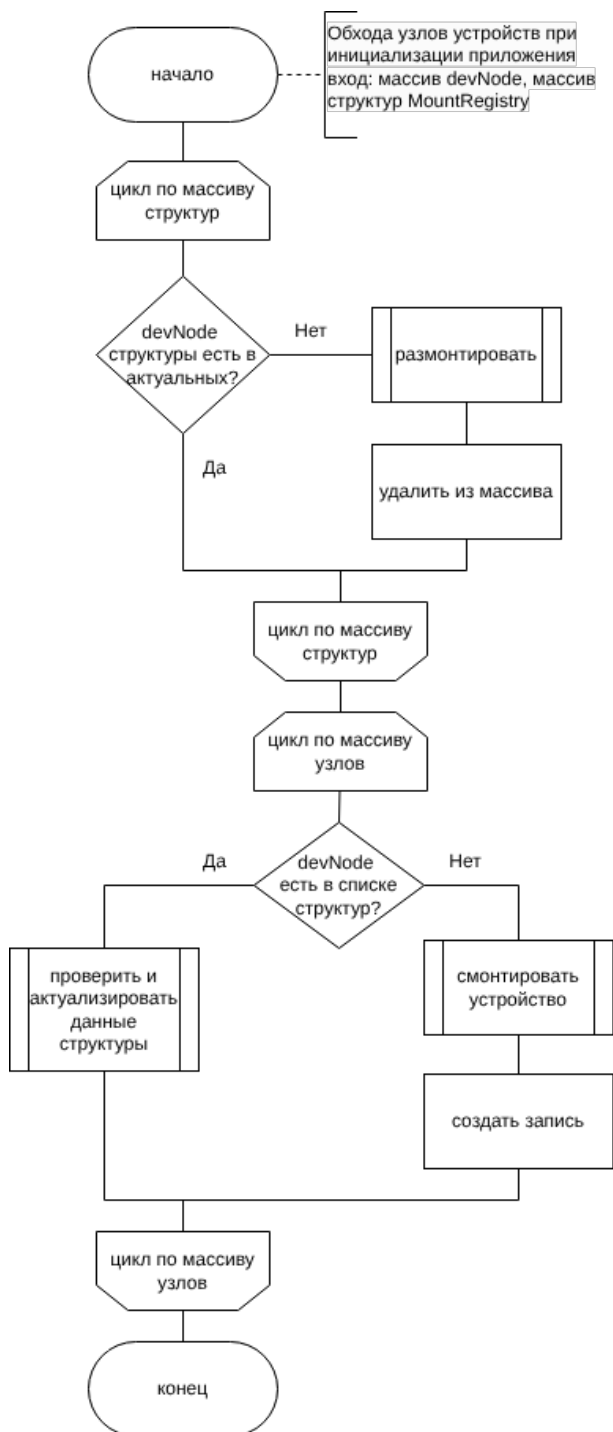
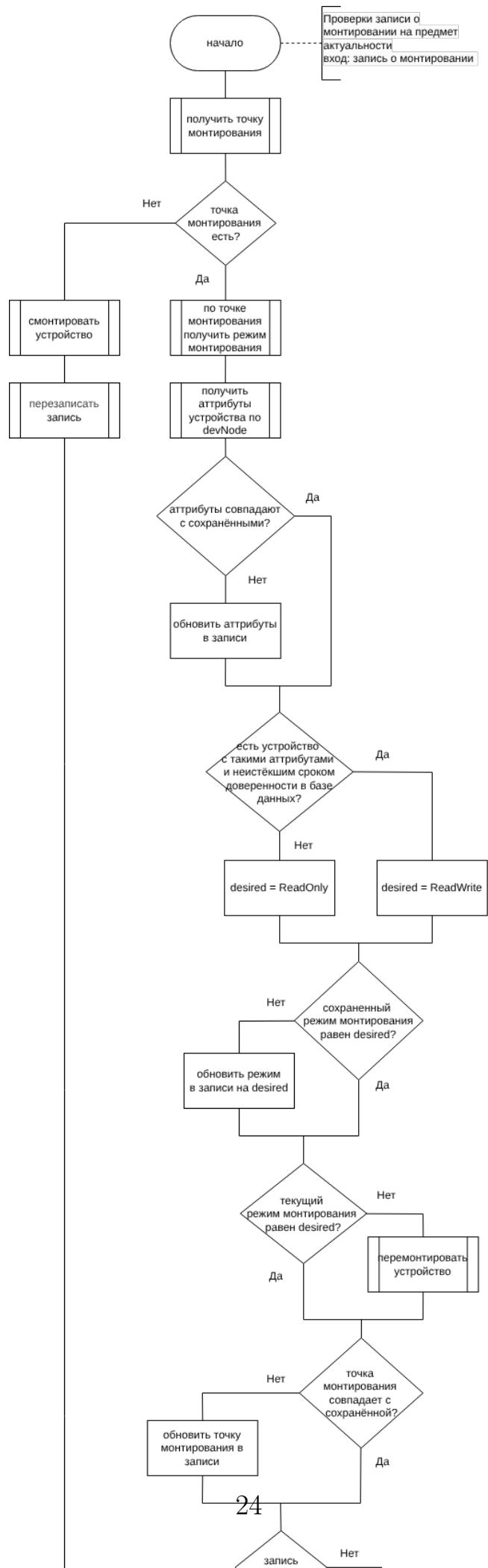


Рисунок 2.5 – Схема алгоритма обработки узлов устройств и записей о точках монтирования при инициализации приложения

## **2.7 Схема алгоритма проверки записи о монтировании на предмет актуальности**

На рисунке 2.6 представлена схема алгоритма проверки записи о монта-  
ровании на предмет актуальности.



## 2.8 Описание сущностей проектируемой базы данных

База данных содержит следующие таблицы:

- `DeviceInfo` — описание устройства:
  1. `id` — первичный ключ;
  2. `vendorId` — идентификатор производителя;
  3. `productId` — идентификатор продукта;
  4. `serial` — серийный номер;
  5. `productName` — название производителя;
  6. `vendorName` — название устройства.
- `Device` — описание записи в списке доверенных устройств:
  1. `id` — первичный ключ;
  2. `deviceId` — идентификатор описания устройства;
  3. `validTo` — дата окончания периода доверия.
- `MountRecord` — описание события монтирования:
  1. `id` — первичный ключ;
  2. `deviceId` — идентификатор описания устройства;
  3. `devNode` — узел устройства;
  4. `mountPoint` — точка монтирования;
  5. `mode` — режим монтирования.

## 2.9 Диаграмма проектируемой базы данных

На рисунке 2.7 представлена диаграмма проектируемой базы данных.

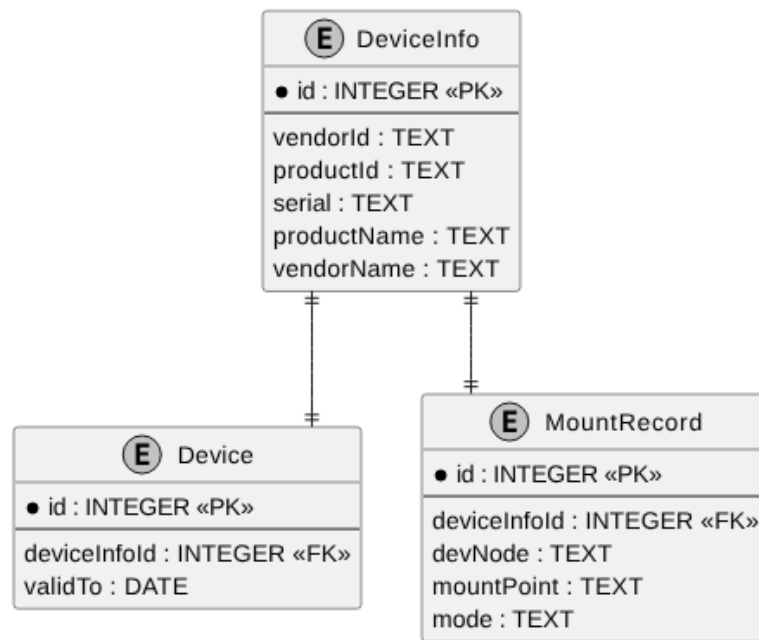


Рисунок 2.7 – Диаграмма базы данных

Между таблицами базы данных образуется связь один к одному.

## 2.10 Описание проектируемых ограничений целостности базы данных

На значения полей базы данных накладываются следующие ограничения целостности:

### — DeviceInfo:

1. поля `vendorId` и `productId` должны содержать только цифры или строчные или заглавные латинские буквы, их длина должна равняться 4-ём символам, они не могут быть пустыми;
2. поле `serial` не может быть пустым;
3. комбинация полей `vendorId`, `productId`, `serial` должна быть уникальной.

### — MountRecord:

1. поле `deviceInfoId` является внешним ключом, ссылающимся на таблицу `DeviceInfo`, должно быть непустым и уникальным;
2. `devNode` должно быть непустым и уникальным;



3. `mountPoint` должно быть непустым и уникальным;
  4. `mode` должно быть непустым и принимать только значения «RO» и «RW».
- **Device:** поле `deviceInfoId` является внешним ключом, ссылающимся на таблицу `DeviceInfo`, должно быть непустым и уникальным, а `validTo` должно содержать дату в формате «ГГГГ-ММ-ДД».

## 3 Технологический раздел

### 3.1 Выбор языка и среды программирования

Для разработки серверной части программно-алгоритмического комплекса был выбран язык программирования C++ [13], что обусловлено следующими факторами:

1. встроенные возможности объектно-ориентированного программирования, позволяющие создавать легко расширяемые приложения;
2. совместимость с языком C, возможность прямого вызова функций мониторинга устройств;
3. поддержка многопоточности, необходимой для параллельной обработки событий;
4. стандартная библиотека предоставляет множество контейнеров, упрощающих разработку.

Для разработки клиентской части был выбран язык TypeScript [14] и фреймворк Vue.js [15]. Использование TypeScript обеспечивает статическую типизацию, что позволяет уменьшить количество ошибок на этапе разработки и повысить удобство сопровождения кода. Кроме того, язык предоставляет современные средства разработки веб-приложений и хорошо интегрируется с большинством популярных frontend-фреймворков.

Выбор Vue.js обусловлен компонентным подходом к разработке пользовательского интерфейса, что упрощает повторное использование элементов приложения и повышает удобство сопровождения проекта. Дополнительно фреймворк предоставляет встроенные средства реактивного обновления интерфейса, позволяющие эффективно отображать изменения состояния приложения в режиме реального времени.

В качестве среды разработки был выбран Visual Studio Code [16] — кроссплатформенная среда, функциональность которой может быть увеличена с использованием дополнительных расширений.

## 3.2 Выбор средств тестирования кода

Для тестирования серверной части приложения был выбран фреймворк GoogleTests [17]. Данный продукт позволяет создавать unit-тесты, e2e-тесты и интеграционные тесты. Разработан специально для C++, и использует стандартный его синтаксис. Позволяет реализовывать Mock-объекты для изоляции зависимостей при тестировании отдельных компонентов системы [18]. Также фреймворк поддерживает параметризованные тесты, что позволяет проверять поведение методов на различных наборах входных данных без дублирования кода тестов. Для выполнения системно-зависимых тестов, таких как монтирование, тесты и приложение запускаются в контейнерах Docker [19].

Для тестирования клиентской части приложения была выбрана библиотека Vitest [20]. Данная библиотека предназначена для тестирования JavaScript- и TypeScript-приложений, а также является нативной для фреймворка Vue.js. Vitest поддерживает создание unit- и компонентных тестов, предоставляет встроенные средства мокирования функций и модулей. Дополнительно библиотека поддерживает параллельное выполнение тестов, что сокращает время тестирования клиентской части приложения.

## 3.3 Обеспечение связи между клиентом и сервером

Для связи клиента и сервера используется протокол HTTP (HyperText Transfer Protocol) [21] и механизм WebSocket [22].

Протокол HTTP является протоколом прикладного уровня модели OSI (Open Systems Interconnection) и предназначен для обмена данными между клиентом и сервером по схеме «запрос-ответ». Клиент формирует HTTP-запрос, сервер обрабатывает его и возвращает ответ, содержащий необходимые данные. Протокол широко применяется при передаче гипертекстовых документов и поддерживается большинством современных программных и аппаратных средств. В разрабатываемой программе используется, например, для получения списка подключённых устройств на клиенте.

Протокол WebSocket используется для организации постоянного двустороннего соединения между клиентом и сервером в режиме реального времени. В отличие от HTTP, WebSocket обеспечивает асинхронный обмен сообщениями и уменьшает накладные расходы на передачу данных за счёт от-

сутствия необходимости повторного создания соединения для каждого запроса. Установление WebSocket-соединения начинается с HTTP-запроса, после чего взаимодействие продолжается по отдельному протоколу WebSocket. В разрабатываемой программе используется для передачи на клиент информации о подключении и извлечении устройств.

### **3.4 Выбор программных средств для работы с устройствами и файловыми системами**

Для взаимодействия с устройствами и файловыми системами в операционной системе Linux были выбраны библиотеки `libudev` [23], `libmount` [24] и `libblkid` [25]. Использование данных библиотек позволяет применять стандартные механизмы Linux для работы с USB-устройствами, операциями монтирования и определением типов файловых систем в пользовательском пространстве.

Библиотека `libudev` используется для получения информации об устройствах и отслеживания событий их подключения и отключения.

В разработанном программном комплексе библиотека `libmount` применяется для чтения файла `/proc/self/mountinfo` и определения режимов монтирования и точек монтирования. Выбор `libmount` обусловлен тем, что библиотека предоставляет унифицированный интерфейс для работы с данными монтирования Linux и позволяет избежать ручного чтения системных файлов.

В разработанной программе библиотека `libblkid` применяется для получения типа файловой системы USB-накопителя перед выполнением операций монтирования.

### **3.5 Выбор встраиваемой системы управления реляционными базами данных (СУБД)**

Для сравнения были выбраны следующие известные системы:

1. SQLite [26];
2. DuckDB [27];
3. H2 Database [28];
4. Apache Derby [29].

Результаты анализа существующих решений приведены в таблице 3.1.

Таблица 3.1 – Результат анализа существующих решений

| <b>Критерий</b>                 | <b>Номер решения</b> |          |          |          |
|---------------------------------|----------------------|----------|----------|----------|
|                                 | <b>1</b>             | <b>2</b> | <b>3</b> | <b>4</b> |
| Поддержка Linux                 | да                   | да       | да       | да       |
| Поддержка C++                   | да                   | да       | нет      | нет      |
| Бесплатное распространение      | да                   | да       | да       | да       |
| Поддержка внешних ключей        | да                   | да       | да       | да       |
| Поддержка ограничений           | да                   | да       | да       | да       |
| Наличие обновлений и поддержки  | да                   | да       | да       | нет      |
| Отсутствие внешних зависимостей | да                   | да       | нет      | нет      |
| Размер библиотеки(Мегабайты)    | 1.5                  | 70       | 2.5      | 6.1      |

На основе анализа, для разрабатываемого комплекса была выбрана SQLite.

### 3.6 Классы эквивалентности

Для разработанного программно-алгоритмического комплекса были созданы основные классы эквивалентности.

#### 3.6.1 Классы эквивалентности для метода построения имени папки монтирования

Класс наличия всех информационных полей устройства:

Входные данные: `vendorId = 1234, productId = 7856, serial = ACDC456IRHX;`

Ожидаемый результат: `/media/dlp/1234_7856_ACDC456IRHX.`

Класс отсутствия у устройства серийного номера:

Входные данные: `vendorId = 1234, productId = 7856;`

Ожидаемый результат: `/media/dlp/1234_7856_noserial.`

Класс отсутствия у устройства идентификаторов производителя и устройства:

Входные данные: `serial = ACDC456IRHX;`

Ожидаемый результат: `/media/dlp/unknown_unknown_ACDC456IRHX.`

### **3.6.2 Классы эквивалентности для обработки события подключения устройства**

Класс подключения недоверенного устройства:

Входные данные: событие подключения устройства `/dev/sda1`, данных о котором нет в таблице Device.

Ожидаемый результат: устройство смонтировано в режиме «Только для чтения».

Класс подключения доверенного устройства:

Входные данные: событие подключения устройства `/dev/sda1`, данные о котором есть в таблице Device и период доверия не истёк.

Ожидаемый результат: устройство смонтировано в режиме полного доступа.

Класс подключения устройства с истёкшим сроком доверия:

Входные данные: событие подключения устройства `/dev/sda1`, данные о котором есть в таблице Device, но период доверия истёк.

Ожидаемый результат: устройство смонтировано в режиме «Только для чтения».

### **3.6.3 Классы эквивалентности для получения записи о монтировании по узлу устройства**

Класс получения существующей записи о монтировании:

Входные данные: в хранилище содержится запись о монтировании для узла `/dev/sda1`.

Ожидаемый результат: запись о монтировании для узла `/dev/sda1`.

Класс получения записи о монтировании для несуществующего узла `/dev/sda1`:

Входные данные: в хранилище отсутствует запись о монтировании для узла `/dev/sda1`.

Ожидаемый результат: нулевой указатель.

### **3.6.4 Классы эквивалентности для получения списка доверенных устройств**

Класс получения заполненного списка доверенных устройств:

Входные данные: в хранилище списка содержится 3 записи об устройствах.

Выходные данные: массив из трёх элементов.

Класс получения пустого списка доверенных устройств:

Входные данные: в хранилище списка отсутствуют записи.

Выходные данные: пустой массив.

### 3.7 Структура проекта серверной части комплекса

Исходный код приложения, а также вспомогательные и служебные элементы были логически разделены по функциональному признаку следующим образом:

- `api-stubs/api/` — содержит в себе обработчики HTTP запросов;
- `external/` — директория для сторонних библиотек. Содержит `ixwebsocket` (библиотека для WebSocket-соединений) и `spdlog` (библиотека для журналирования);
- `sql/schema` — хранит скрипты для создания таблицы базы данных;
- `src/` — исходный код приложения:
  - `commands/` — содержит классы команд:
    - `Command` — базовый класс команды;
    - `GetWhiteListDeviceCommand` — команда получения списка доверенных устройств;
    - `AddDeviceToWhiteListCommand` — команда добавления устройства в список доверенных;
    - `DeleteDeviceFromWhiteListCommand` — команда удаления устройства из списка доверенных;
    - `PatchValidToDeviceCommand` — команда изменения даты истечения доверия устройства;
    - `GetCurrentConnectedDevicesCommand` — команда получения списка подключённых в данный момент устройств;
    - `CommandContext` — вспомогательный класс, содержащий необходимые сервисы для команд.

- **core/** — содержит классы, реализующие основную работу по монтированию устройств и обработки событий:
  - **DeviceControlService** — класс управляет обработкой события;
  - **EventLoop** — класс, отправляющий события из очереди на обработку;
  - **EventQueue** — класс, управляющий состоянием очереди событий;
  - **IDeviceResolver** — интерфейс класса получения параметров устройств и монтирования;
  - **IMountSystem** — интерфейс класса операций монтирования/размонтирования;
  - **LinuxMountSystem** — класс операций монтирования/размонтирования;
  - **MountRecoveryService** — класс обновления записей о монтировании при запуске приложения;
  - **MountService** — класс, управляющий операциями монтирования;
  - **MountUtils** — класс-обёртка над операциями монтирования/размонтирования;
  - **UdevDeviceResolver** — класс получения параметров устройств и монтирования;
  - **UdevWatcher** — класс, отслеживающий события с устройствами.
- **essesnses/** — содержит классы-сущности и их классы породители:
  - **Device** — класс устройства;
  - **DeviceBuilder** — класс-создатель объекта устройства;
  - **DeviceEvent** — класс события подключения/отключения устройства;
  - **DeviceEventBuilder** — класс-создатель объекта события;
  - **DeviceInfo** — класс информации об устройстве;
  - **DeviceInfoBuilder** — класс-создатель объекта информации;



- **MountRecord** — класс записи о монтировании;
- **MountRecordBuilder** — класс-создатель объекта записи о монтировании.
- **facade/** — содержит класс **Facade**, унифицированный интерфейс доступа к подсистемам;
- **managers/** — содержит классы **DeviceManager** и **MountRegistryManager**, предоставляющие доступ к классам репозитория;
- **repositories/** — содержит классы для работы с базой данных:
  - **DBConnection** — реализует подключение к базе данных, обёртку над функциями выполнения запросов;
  - **DeviceRepository** — реализует работу с таблицами **Device** и **DeviceInfo** базы данных;
  - **MountRepository** — реализует работу со списком записей о монтировании.
- **services/** — содержит вспомогательные классы:
  - **Config** — класс чтения конфигурации приложения;
  - **DBInitializer** — класс инициализации базы данных;
  - **DeviceEventNotifier** — класс отправки сообщений о событиях устройств на клиент;
  - **DevLogger** — класс логгера;
  - **BaseException** — базовый класс исключения;
  - **FileException** — класс исключения, связанного с файловой работой;
  - **HttpServerError** — класс исключения, связанного с HTTP сервером;
  - **SqlDataBaseError** — класс исключения, связанного с работой с базой данных;
  - **ResolveInfoError** — класс исключения, связанного с невозможностью извлечь данные об устройстве;
  - **UnknownFsError** — класс исключения, связанного с невозможностью установить тип файловой системы устройства;

- `MountError` — класс исключения, связанного с операцией монтирования;
- `UnmountError` — класс исключения, связанного с операцией размонтирования;
- `HttpServer` — класс запуска HTTP сервера;
- `IWebSocketServer` — интерфейс класса работы с WebSocket;
- `MountPointBuilder` — класс создания пути монтирования;
- `WebSocketServer` — класс работы с WebSocket.
- `main.cpp` — точка входа в программу.
- `tests/` — папка с тестами GoogleTests:
  - `e2e/` — End-To-End тесты;
  - `helpers/` — вспомогательные функции и классы, используемые в тестовом коде;
  - `integration/` — интеграционные тесты;
  - `mocks/` — мокированные объекты для тестов;
  - `unit/` — юнит-тесты;
  - `CMakeLists.txt` — сценарий сборки тестового кода.
- `CMakeLists.txt` — сценарий сборки приложения;
- `docker-compose.yml` — конфигурационный файл Docker;
- `Dockerfile.app` — инструкции для создания образа контейнера приложения;
- `Dockerfile.e2e` — инструкции для создания образа контейнера для e2e тестов;
- `Dockerfile.tests` — инструкции для создания образа контейнера для юнит- и интеграционных тестов.

## 3.8 Листинги ключевых методов, классов и функций

В листинге 3.1 представлена точка входа в приложение.

Листинг 3.1 – Точка входа в приложение

```
#include "Facade.hpp"
#include "DBInitializer.hpp"
#ifdef BUILD_HTTP_SERVER
#include "HttpServer.hpp"
#endif
#include "EventQueue.hpp"
#include "Watcher.hpp"
#include "EventLoop.hpp"
#include "Config.hpp"
#include "LinuxMountSystem.hpp"
#include "UdevDeviceResolver.hpp"
#include "MountRecoveryService.hpp"
#include "WebSocketServer.hpp"
#include "DeviceEventNotifier.hpp"

#include <thread>
#include <iostream>
#include <chrono>

class App {
private:
    DBConnection db;
    Facade facade;
#ifdef BUILD_HTTP_SERVER
    HttpServer http;
#endif
    LinuxMountSystem linms;
    UdevDeviceResolver resolver;
    MountRecoveryService rec;

public:
    App()
        : db(Config::getDBPath()),
          facade(db, linms, resolver),
#ifdef BUILD_HTTP_SERVER
          http(facade),
#endif
    }
```

```

        rec(facade.registry(), resolver, facade.mounts())
    {
        DBInitializer::init(db);
    }

void run()
{
    rec.run();

    WebSocketServer ws(Config::getWebSocketPort());
    ws.start();
    DeviceEventNotifier notifier(ws);

    EventQueue<DeviceEvent> queue;

    UdevWatcher watcher(queue);

    DeviceControlService service(
        facade.registry(),
        facade.mounts(),
        notifier
    );

    EventLoop loop(queue, service);

    std::jthread watcherThread([&] {
        try {
            watcher.run();
        }
        catch (const std::exception& e) {
            std::cerr << "watcher thread: "
                << e.what()
                << std::endl;
        }
    });

    std::jthread loopThread([&] {
        try {
            loop.run();
        }
        catch (const std::exception& e) {

```

```

        std::cerr << "loop thread: "
                    << e.what()
                    << std::endl;
    }
});

#ifdef BUILD_HTTP_SERVER
    std::jthread httpThread([&] {
        try {
            http.start();
        }
        catch (const std::exception& e) {
            std::cerr << "http thread: "
                        << e.what()
                        << std::endl;
        }
    });
#endif

    for (;;) {
        std::this_thread::sleep_for(
            std::chrono::seconds(10));
    }
}

};

int main() {
    App app;
    app.run();
}

```

В листинге 3.2 представлен метод класса DeviceControlService, отвечающий за обработку события подключения/отключения устройства.

Листинг 3.2 – Метод обработки события устройства

```

void handleEvent(const DeviceEvent& event)
{
    mylog->info("Start handle {} event with devnode {}",
               event.type == EventType::INSERT ? "INSERT" : "REMOVE",
               event.devNode);
    if (event.type == EventType::INSERT) {
        auto record = mountManager_.mount(event.devNode);
    }
}

```

```

        mountRegistry_.add(*record);
        notifier_.notifyInsert(*record);
    }
    else if (event.type == EventType::REMOVE) {
        if (std::optional<std::string> mountPoint =
            mountRegistry_.getMountPointByDevNode(event.devNode))
        {
            mountManager_.unmount(*mountPoint);
            notifier_.notifyRemove(*mountPoint);
            mountRegistry_.removeByDevNode(event.devNode);
        }
        else {
            mylog->warn("No mount point for {}", event.devNode);
        }
    }
}
}

```

В листинге 3.3 представлен класс MountRecoveryService, отвечающий за актуализацию записей о монтировании.

Листинг 3.3 – Класс актуализации записей о монтировании

```

#pragma once
#include <vector>
#include <string>
#include <optional>
#include <unordered_set>
#include "MountRecord.hpp"
#include "MountRegistry.hpp"
#include "MountService.hpp"
#include "IDeviceResolver.hpp"

class MountRecoveryService {
private:
    MountRegistryManager &registry;
    IDeviceResolver &resolver;
    MountService &manager;
public:
    explicit MountRecoveryService(
        MountRegistryManager& mr,
        IDeviceResolver &rs,
        MountService& man)
        : registry(mr),

```

```

        resolver(rs),
        manager(man) {}

void actualize(MountRecord &rec) {
    const auto& devNode = rec.devNode;
    auto info = resolver.resolve(devNode.c_str());
    if (!info) return;
    const auto mountPoint =
        resolver.getMountPoint(devNode);
    if (!mountPoint) {
        auto newrec = manager.mount(devNode);
        if (newrec) registry.recreate(*newrec);
        return;
    }
    auto md = resolver.getMountMode(*mountPoint);
    bool modeChanged = md != rec.mode;
    bool infoChanged = *info != rec.info;
    if (modeChanged || infoChanged) {
        auto newrec = manager.remount(rec);
        if (newrec) registry.recreate(*newrec);
        return;
    }
    if (*mountPoint != rec.mountPoint) {
        rec.mountPoint = *mountPoint;
        registry.refresh(rec);
    }
}

void run()
{
    auto regs = registry.getAll();
    auto devNodes = resolver.getUsbDevNodes();
    std::unordered_map<std::string, MountRecord> regMap;
    for (const auto& reg : regs)
        regMap[reg.devNode] = reg;
    std::unordered_set<std::string> devSet(
        devNodes.begin(),
        devNodes.end());
    for (const auto& reg : regs) {
        if (!devSet.contains(reg.devNode)) {
            registry.removeByDevNode(reg.devNode);

```

```

        manager.unmount(reg.mountPoint);
    }
}
for (const auto& node : devNodes) {
    auto it = regMap.find(node);
    if (it != regMap.end()) {
        actualize(it->second);
    }
    else {
        auto rec = manager.mount(node);
        if (rec)
            registry.add(*rec);
    }
}
}
};

```

### 3.9 Развёртывание приложения

Запуском серверной части управляет systemd [30]. Файл конфигурации systemd располагается по пути `/etc/systemd/system/usb-man.service`, его содержимое приведено в листинге 3.4.

Листинг 3.4 – Конфигурация systemd

```

[Unit]
Description=USBMan

[Service]
ExecStart=/usr/local/bin/usbman
Restart=always

[Install]
WantedBy=multi-user.target

```

Исполняемый файл сервера находится по пути `/usr/local/bin/usbman`, файл базы данных лежит в `/var/lib/usbman/app.db`, SQL-скрипты для создания таблиц находятся по пути `/usr/share/usbman/sql/`, файл конфигурации для приложения находится в `/etc/usbman/config.txt`, его содержимое приведено в листинге 3.5.

Листинг 3.5 – Конфигурация приложения



```
http.port=8080
websocket.port=9000
db.schema=/usr/share/usbman/sql/schema/001_device_info.sql;
        /usr/share/usbman/sql/schema/002_device.sql;
        /usr/share/usbman/sql/schema/003_mount_record.sql
db.path=/var/lib/usbman/app.db
```

Запуск клиентского приложения выполняется с помощью `systemd` и сервера `Nginx` [31]. Конфигурация `Nginx` располагается в файле `/etc/nginx/sites-available/usb-ui`, её содержимое приведено в листинге 3.6.

Листинг 3.6 – Конфигурация `Nginx`

```
server {
    listen 5173;

    root /var/www/usb-ui;
    index index.html;

    location / {
        try_files $uri /index.html;
    }

    location /api/ {
        proxy_pass http://127.0.0.1:8080;
    }

    location /ws {
        proxy_pass http://127.0.0.1:9000;

        proxy_http_version 1.1;

        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";
    }
}
```

Файлы, отдаваемые `Nginx` лежат в папке `/var/www/usb-ui`. В ней находятся собранные `*.html`, `*.js`, `*.css` и графические файлы.

## 3.10 Взаимодействие пользователя с приложением

Пользователь может взаимодействовать с программным комплексом через Web-интерфейс, доступный по адресу <http://localhost:5173/>.

### 3.10.1 Экран подключённых устройств

По адресу <http://localhost:5173/list/> пользователю доступен список подключённых в данный момент устройств. Над список есть 2 кнопки для перехода по экранам приложения. Слева находится кнопка с надписью «Подключённые устройства» (переход на описываемый экран), а справа от неё кнопка с надписью «Доверенные устройства» (переход в список доверенных устройств). Для каждого устройства отображаются следующие данные:

1. путь монтирования;
2. режим монтирования;
3. название производителя;
4. название устройства;
5. серийный номер;
6. идентификатор производителя;
7. идентификатор устройства.

Также справа от каждого устройства отображается иконка добавления в список доверенных устройств с изображением знака +, при наведении указателя мыши на которую всплывает подсказка с текстом «Добавить». В случае, если устройство уже доверено, справа отображается иконка с изображением знака ✓, при наведении курсора на которую всплывает подсказка с текстом «Уже добавлено». При наведении указателя мыши на любую из кнопок курсор меняется на символ . Макет описанного экрана представлен на рисунке 3.1.

Подключённые устройства

Доверенные устройства

#### Подключённые устройства

| Путь монтирования | Режим         | Производитель | Имя устройства   | Серийный номер | Идентификатор производителя | Идентификатор продукта |   |
|-------------------|---------------|---------------|------------------|----------------|-----------------------------|------------------------|---|
| /media/usb1       | Только чтение | Kingston      | DataTraveler 3.0 | DTX-001245     | 0951                        | 1666                   | + |
| /media/usb2       | Полный доступ | SanDisk       | Ultra USB        | SDK-882341     | 0781                        | 5581                   | ✓ |
| /media/usb3       | Только чтение | Transcend     | JetFlash         | TR-771245      | 8564                        | 1000                   | + |
| /media/usb4       | Полный доступ | Samsung       | BAR Plus         | SM-552341      | 04E8                        | 61B6                   | ✓ |
| /media/usb5       | Только чтение | ADATA         | UV150            | AD-982341      | 125F                        | C93A                   | + |
| /media/usb6       | Полный доступ | Apacer        | AH333            | AP-112233      | 1005                        | B113                   | ✓ |
| /media/usb7       | Только чтение | Silicon Power | Blaze B05        | SP-554433      | 13FE                        | 5500                   | + |
| /media/usb8       | Полный доступ | Verbatim      | Store n Go       | VB-765421      | 18A5                        | 0243                   | ✓ |

Рисунок 3.1 – Экран подключённых в данный момент устройств

### 3.10.2 Экран доверенных устройств

По адресу <http://localhost:5173/whitelist/> пользователю доступен список доверенных устройств. Над списком аналогично находятся кнопки навигации. Для каждого устройства отображаются следующие параметры:

1. название производителя;
2. название устройства;
3. серийный номер;
4. идентификатор производителя;
5. идентификатор устройства;
6. дата истечения доверия.

В случае если дата истечения доверия не задана (устройство доверено бессрочно), то в поле «Доверено до» отображается шаблон даты **дд.мм.гггг.**

Также справа от каждого устройства отображается иконка удаления из списка доверенных устройств с изображением символа , при наведении указателя мыши на которую всплывает подсказка с текстом «Удалить».

Макет описанного экрана представлен на рисунке 3.2.

Подключённые устройства
Доверенные устройства

**Доверенные устройства**

| Производитель | Имя устройства   | Серийный номер | Идентификатор производителя | Идентификатор продукта | Доверено до |   |    |
|---------------|------------------|----------------|-----------------------------|------------------------|-------------|---|----|
| Kingston      | DataTraveler 3.0 | DTX-001245     | 0951                        | 1666                   | 31.12.2026  | 📅 | 🗑️ |
| SanDisk       | Ultra USB        | SDK-882341     | 0781                        | 5581                   | ДД.ММ.ГГГГ  | 📅 | 🗑️ |
| Samsung       | BAR Plus         | SM-552341      | 04E8                        | 61B6                   | 10.05.2027  | 📅 | 🗑️ |
| ADATA         | UV150            | AD-982341      | 125F                        | C93A                   | ДД.ММ.ГГГГ  | 📅 | 🗑️ |
| Corsair       | Flash Voyager    | CR-998877      | 1B1C                        | 1A90                   | 15.08.2026  | 📅 | 🗑️ |
| PNY           | Turbo Attaché    | PN-334455      | 1548                        | 6545                   | 01.01.2028  | 📅 | 🗑️ |
| Transcend     | JetFlash         | TR-771245      | 8564                        | 1000                   | ДД.ММ.ГГГГ  | 📅 | 🗑️ |
| Verbatim      | Store n Go       | VB-765421      | 18A5                        | 0243                   | 20.11.2027  | 📅 | 🗑️ |
| Silicon Power | Blaze B05        | SP-554433      | 13FE                        | 5500                   | ДД.ММ.ГГГГ  | 📅 | 🗑️ |
| Apacer        | AH333            | AP-112233      | 1005                        | B113                   | 30.09.2026  | 📅 | 🗑️ |

Рисунок 3.2 – Экран доверенных устройств

При нажатии на поле «Доверено до» появляется всплывающее окно календаря, в котором пользователь может указать дату истечения доверия, которая не может быть раньше текущей. В случае, если необходимо сделать доверие бессрочным, можно нажать на кнопку календаря с надписью «Удалить». Макет выбора даты представлен на рисунке 3.3.

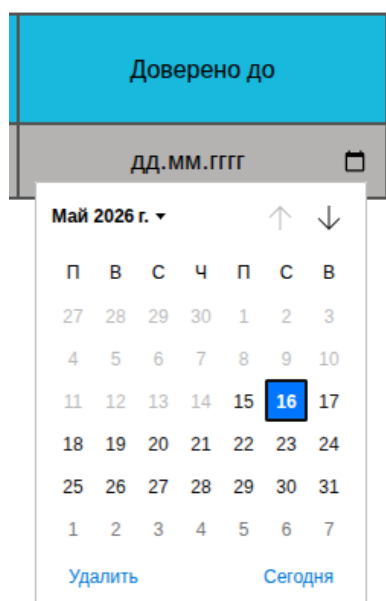


Рисунок 3.3 – Экран выбора даты истечения доверия в календаре

## 4 Исследовательский раздел

В данном разделе проводится исследование временных характеристик разработанного программного обеспечения.

### 4.1 Постановка исследования

Предметом исследования является время выполнения алгоритма обработки узлов устройств и записей о точках монтирования при инициализации приложения при различных начальных условиях. В первой части исследования меняется количество записей о точках монтировании, а во второй части меняется количество актуальных записей о точках монтирования.

### 4.2 Технические характеристики

Исследование проводилось на виртуальной машине с операционной системой Linux Ubuntu Server. Для создания виртуальной машины использовался программный механизм KVM (Kernel-based Virtual Machine) [32].

Конфигурация виртуальной машины:

- количество процессоров — 4;
- объём оперативной памяти — 6 гигабайт;
- объём виртуального диска — 20 гигабайт.

Хостовая машина работает на процессоре 11th Gen Intel(R) Core(TM) i7-11370H @ 3.30 ГГц [33].

### 4.3 Проведение исследования и результаты

Для проведения исследования с помощью модуля ядра `dummy_hcd` [34] было создано виртуальное устройство USB с восьмью LUN (Logical Unit Number).

#### 4.3.1 План первой части исследования

Количество исходных записей о монтировании  $N$  изменяется от 0 до 8. Для каждого состояния проводится 50 замеров времени и их дальнейшее

усреднение. Перед каждым замером очищается хранилище записей о монтаже, все устройства размонтируются, N устройств из 8 монтируется, записи о них сохраняются. Количество виртуальных устройств неизменно.

### 4.3.2 Результаты первой части исследования

В результате был получен график зависимости времени выполнения алгоритма обработки узлов устройств и записей о точках монтирования от количества исходных записей о точках монтирования, представленный на рисунке 4.1.

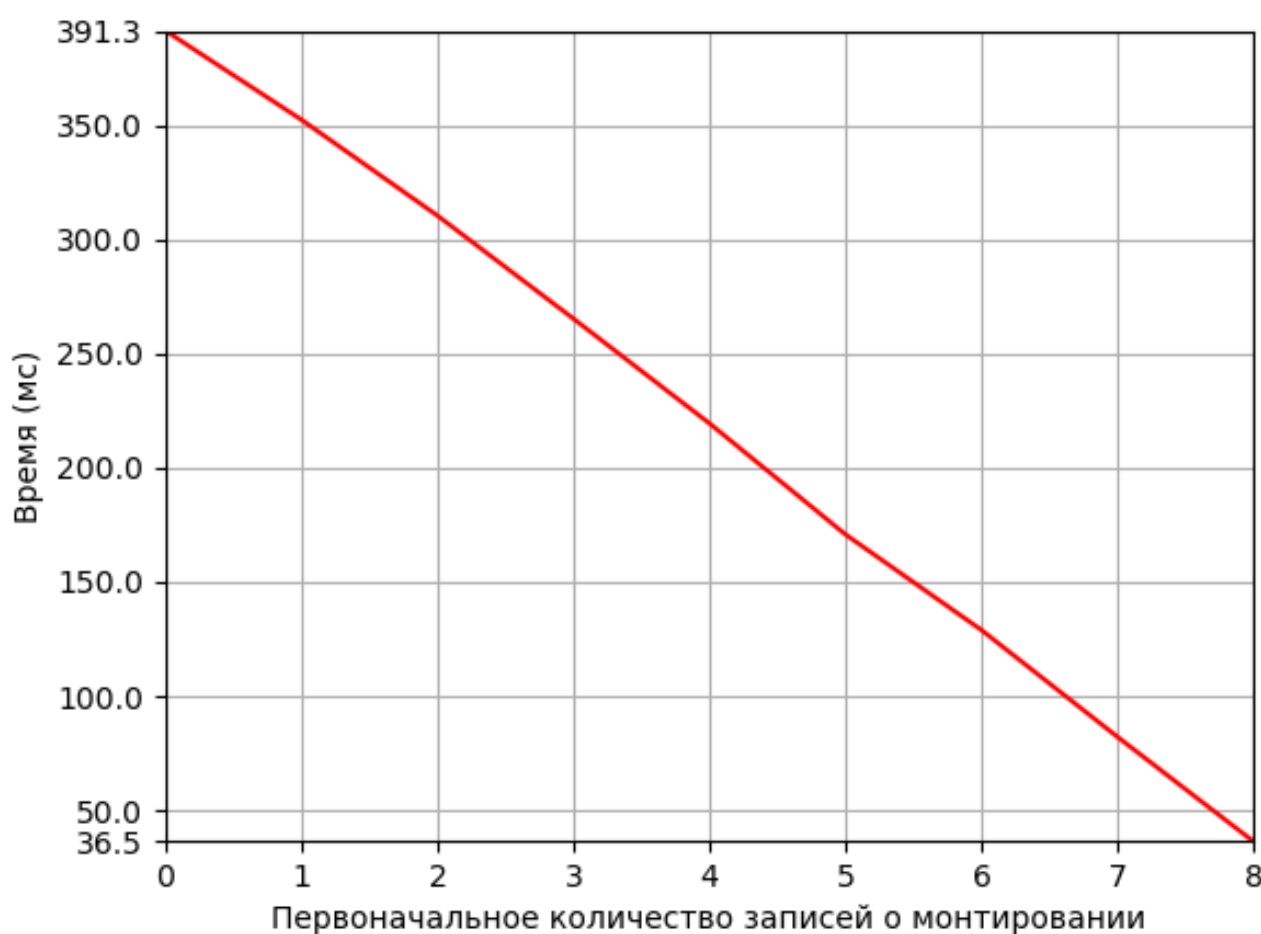


Рисунок 4.1 – Результат первой части исследования

Время выполнения алгоритма практически линейно зависит от количества записей. При наличии всех записей о монтаже время выполнения уменьшается более чем в 10 раз относительно состояния отсутствия записей.

### 4.3.3 План второй части исследования

Количество исходных записей о монтировании и количество виртуальных устройств неизменно и равно 8. Для каждого состояния проводится 50 замеров времени и их дальнейшее усреднение. Перед каждым замером очищается хранилище записей о монтировании, все устройства перемонтируются, часть записей сохраняется в достоверном виде, а  $N$  записей записи сохраняются в изменённом формате — режим монтирования меняется на противоположный или меняется путь монтирования, где  $N$  изменяется от 0 до 8.

### 4.3.4 Результаты второй части исследования

В результате был получен график зависимости времени выполнения алгоритма обработки узлов устройств и записей о точках монтирования от количества недостоверных записей о монтировании, представленный на рисунке 4.2.

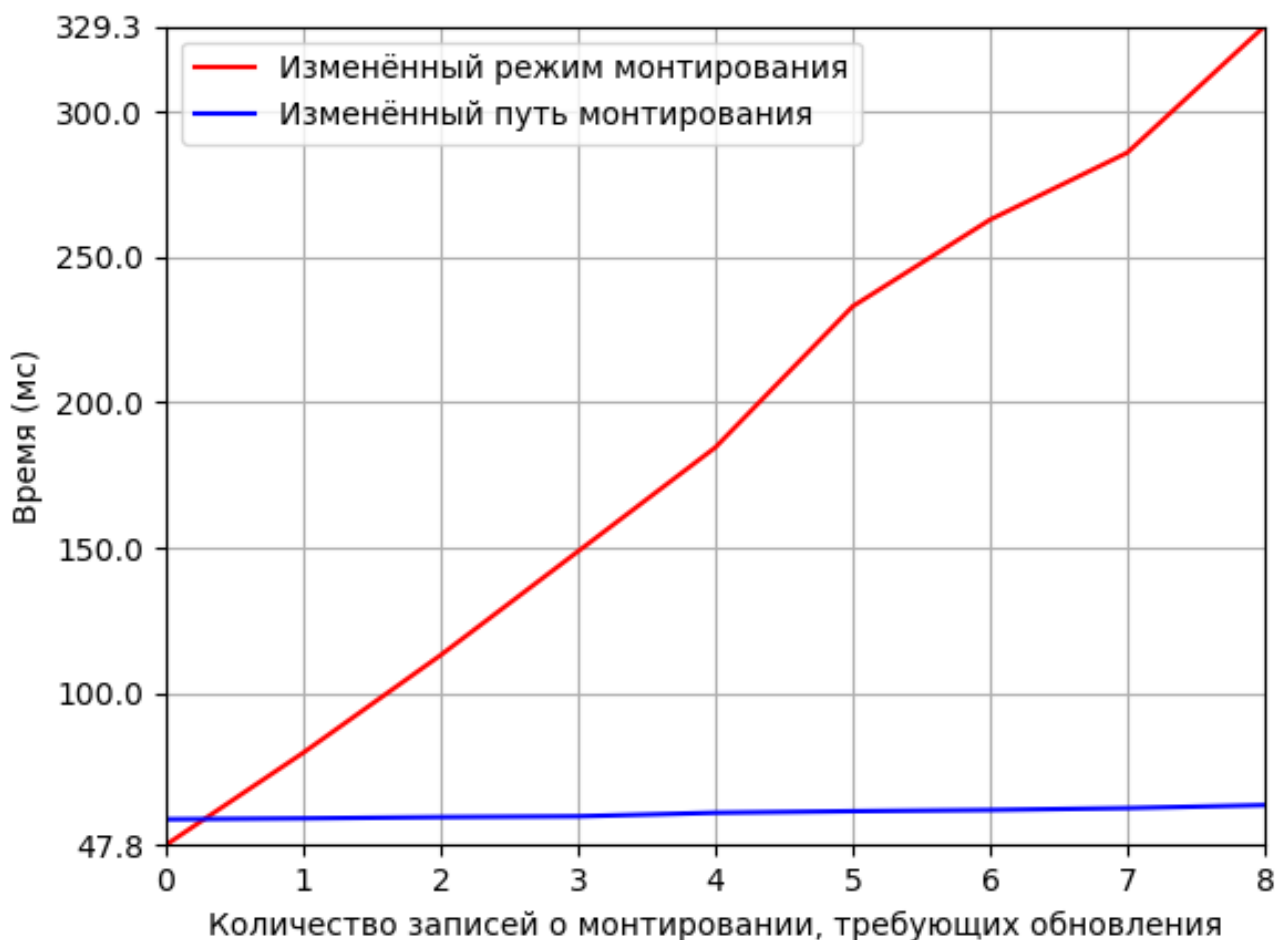


Рисунок 4.2 – Результат второй части исследования

При обнаружении несоответствия фактического режима монтирования и документированного режима происходит перемонтирование и обновление записи о монтировании, что отражается на росте времени выполнения алгоритма. В случае изменённых режимов монтирования наблюдается семикратное увеличение времени выполнения при недостоверности всех 8 записей относительно полностью правильного набора.

При обнаружении несоответствия фактического пути монтирования и документированного пути происходит только обновление записи о монтировании, что практически никак не отражается на времени выполнения алгоритма.



## ЗАКЛЮЧЕНИЕ

## СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Баяв Алексей Владимирович, Гаценко Олег Юрьевич, Самонов Александр Валерьянович Программный комплекс управления доступом USB-устройств к автоматизированным рабочим местам // Вопросы кибербезопасности. 2020. №1 (35).
2. InfoWatch — Аналитика информационной безопасности; [Электронный ресурс]. — Режим доступа: <https://www.infowatch.ru/analytics> (дата обращения: 01.03.2026).
3. USBGuard; [Электронный ресурс]. — Режим доступа: <https://usbguard.github.io/> (дата обращения: 01.03.2026).
4. Making USB Great Again with USBFILTER — a USB layer firewall in the Linux kernel; [Электронный ресурс]. — Режим доступа: <https://davejingtian.org/2016/08/04/making-usb-great-again-with-usbfilter-a-usb-layer-firewall-in-the-linux-kernel/> (дата обращения: 01.03.2026).
5. USBWall Source Code; [Электронный ресурс]. — Режим доступа: <https://github.com/usbwall/usbwall?ysclid=mndivt3urh835120502> (дата обращения: 01.03.2026).
6. USBShield Source Code; [Электронный ресурс]. — Режим доступа: <https://github.com/USBShield/USBShield?ysclid=mndiyaq4ah326619085> (дата обращения: 01.03.2026).
7. Sordum — Ratoool v1.4 (Removable Access tool); [Электронный ресурс]. — Режим доступа: <https://www.sordum.org/8104/ratoool-v1-4-removable-access-tool/> (дата обращения: 01.03.2026).
8. Gilisoft USB Lock — Control USB ports, removable devices, and trusted-device access to reduce data leakage on Windows; [Электронный ресурс]. — Режим доступа: <https://www.gilisoft.com/product-usb-lock.htm> (дата обращения: 01.03.2026).
9. NewSoftwares — USBBlock. Restrict Access of Portable Drives; [Электронный ресурс]. — Режим доступа: <https://www.newsoftwares.net/usb-block/> (дата обращения: 01.03.2026).

10. Модели баз данных: учебное пособие / О.Е. Аврунев, В.М. Стасышин. — Новосибирск: Изд-во НГТУ, 2018. — 124 с.
11. Создание базы данных для разработки облачной платформы хранения и редактирования 3D моделей конфет / В. Г. Благовещенский, А. Е. Краснов, И. Г. Благовещенский [и др.] // Интеллектуальные автоматизированные управляющие системы в биотехнологических процессах : сборник докладов всероссийской научно-практической конференции, Москва, 29 марта 2023 года. — Москва: РОССИЙСКИЙ БИОТЕХНОЛОГИЧЕСКИЙ УНИВЕРСИТЕТ; ЗАО «Университетская книга», 2023. — С. 85-96. — EDN NAJYYS.
12. OpenAPI Initiative — The OpenAPI Initiative; [Электронный ресурс]. — Режим доступа: <https://www.openapis.org/> (дата обращения: 01.03.2026).
13. Standard C++; [Электронный ресурс]. — Режим доступа: <https://isocpp.org/> (дата обращения: 01.03.2026).
14. TypeScript — JavaScript With Syntax For Types; [Электронный ресурс]. — Режим доступа: <https://www.typescriptlang.org/> (дата обращения: 01.03.2026).
15. Vue.js — The Progressive JavaScript Framework | Vue.js; [Электронный ресурс]. — Режим доступа: <https://vuejs.org/> (дата обращения: 01.03.2026).
16. Visual Studio Code — The open source AI code editor; [Электронный ресурс]. — Режим доступа: <https://code.visualstudio.com/> (дата обращения: 01.03.2026).
17. GoogleTest User's Guide; [Электронный ресурс]. — Режим доступа: <https://google.github.io/googletest/> (дата обращения: 01.03.2026).
18. Тюменцев, Е. А. Автоматизированное тестирование сложности алгоритмов с помощью Mock-объектов / Е. А. Тюменцев // Математические структуры и моделирование. — 2013. — № 1(27). — С. 82-88. — EDN QLJADF.
19. Docker — Accelerated Container Application Development; [Электронный ресурс]. — Режим доступа: <https://www.docker.com/> (дата обращения: 01.03.2026).

20. Vitest — Next Generation testing framework; [Электронный ресурс]. — Режим доступа: <https://vitest.dev/> (дата обращения: 01.03.2026).
21. Борисов П. Е. HTTP-протокол прикладного уровня (обзор) // Научный аспект. — 2024. — Т. 22. — №. 5. — С. 2937-2943.
22. Хабаров, С. П. Взаимодействия узлов сети по протоколу WebSocket / С. П. Хабаров // Информационные системы и технологии: теория и практика : Сборник научных трудов научно-технической конференции института леса и природопользования, Санкт-Петербург, 01 февраля 2017 года. Том Выпуск 9. — Санкт-Петербург: Санкт-Петербургский государственный лесотехнический университет им. С.М. Кирова, 2017. — С. 104-115. — EDN YRLWFS.
23. libudev(3) — Linux manual page; [Электронный ресурс]. — Режим доступа: <https://man7.org/linux/man-pages/man3/libudev.3.html> (дата обращения: 01.03.2026).
24. The Linux Kernel documentation — libmount Reference Manual; [Электронный ресурс]. — Режим доступа: <https://www.kernel.org/pub/linux/utils/util-linux/v2.36/libmount-docs/index.html> (дата обращения: 01.03.2026).
25. libblkid(3) — Linux manual page; [Электронный ресурс]. — Режим доступа: <https://man7.org/linux/man-pages/man3/libblkid.3.html> (дата обращения: 01.03.2026).
26. SQLite Home Page; [Электронный ресурс]. — Режим доступа: <https://www.sqlite.org/> (дата обращения: 01.03.2026).
27. DuckDB — An in-process SQL OLAP database management system; [Электронный ресурс]. — Режим доступа: <https://duckdb.org/> (дата обращения: 01.03.2026).
28. H2 Database Engine — Welcome to H2, the Java SQL database; [Электронный ресурс]. — Режим доступа: <https://h2database.com/html/main.html> (дата обращения: 01.03.2026).
29. Apache Derby; [Электронный ресурс]. — Режим доступа: <https://db.apache.org/derby/> (дата обращения: 01.03.2026).

30. Systemd — System and Service Manager; [Электронный ресурс]. — Режим доступа: <https://systemd.io/> (дата обращения: 01.03.2026).
31. nginx; [Электронный ресурс]. — Режим доступа: <https://nginx.org/ru/> (дата обращения: 01.03.2026).
32. KVM — Main Page; [Электронный ресурс]. — Режим доступа: [https://linux-kvm.org/page/Main\\_Page](https://linux-kvm.org/page/Main_Page) (дата обращения: 01.03.2026).
33. Intel Core i7-11370H Specs — TechPowerUp CPU Database; [Электронный ресурс]. — Режим доступа: <https://www.techpowerup.com/cpu-specs/core-i7-11370h.c2833> (дата обращения: 01.03.2026).
34. Linux source code (v7.0.8) — Bootlin Elixir Cross Referencer; [Электронный ресурс]. — Режим доступа: [https://elixir.bootlin.com/linux/v6.7-rc3/source/drivers/usb/gadget/udc/dummy\\_hcd.c#L82](https://elixir.bootlin.com/linux/v6.7-rc3/source/drivers/usb/gadget/udc/dummy_hcd.c#L82) (дата обращения: 01.03.2026).

## ПРИЛОЖЕНИЕ А